Clase Warehouse

Warehouse modela el almacén como una grilla, valida la ubicación de sus elementos y genera los grafos dirigidos que utiliza la navegación. Completa el espacio libre con pasillos y comprueba que los productos puedan recorrerse desde y hacia las zonas de despacho.

1. Construcción del almacén (domain.Warehouse.Warehouse)

Idea

Centralizar el layout y las entidades operativas en un único objeto. Cada incorporación mantiene índices de ocupación para detectar conflictos; luego, la clase traduce las celdas transitables en conexiones y comprime tramos rectos para facilitar la planificación de rutas.

Estructuras del código

- Diccionarios de entidades: puertas, zonas de despacho, racks, pasillos, pickers, productos y pedidos, indexados por identificador.
- Índices de celdas: _rack_cell, _aisle_cell y _dispatch_zone_cell registran ocupación; _puntos_acc reserva accesos a racks y _acceso_cargados reúne accesos de productos.
- Conjunto celdas: contiene un Vertice por celda de pasillo y permite consultar su coordenada y pasillo propietario.
- Grafo completo: relaciona cada coordenada transitable con sus vecinos ortogonales dirigidos y costos de un paso.
- Grafo reducido: conserva accesos y puntos de decisión; cada arista representa un tramo recto y guarda su distancia.

Paso a paso

1. El constructor valida ancho_x y largo_y como enteros positivos, inicializa las colecciones e índices y registra las entidades recibidas mediante sus métodos add_*.
2. Las altas validan tipo, ID único, límites y compatibilidad espacial. Una zona requiere una puerta existente y adyacente; un producto requiere un rack existente. Por eso las referencias deben incorporarse en ese orden.
3. build_graph llama a generar_pasillos_faltantes. La grilla que queda libre de racks, zonas y pasillos se particiona en rectángulos que se registran como pasillos automáticos de doble sentido.
4. Para cada celda de pasillo, build_graph considera sus cuatro vecinos. Si ambos pasillos permiten el sentido del movimiento, agrega una arista de costo 1. Las transiciones entre pasillos actualizan sus listas de entrada y salida.
5. build_reduced_graph_v2 conserva accesos de productos, esquinas, contactos entre pasillos y celdas exteriores a las zonas de despacho. Desde esos puntos sigue las aristas dirigidas y conecta el siguiente punto de decisión con el costo del tramo.
6. bloquear_pasillos_sin_salida detecta sectores alcanzables desde una entrada que no tienen continuación hasta una salida. Los quita de los dos grafos y de los índices de circulación; repite el análisis porque una eliminación puede generar otra sección atrapada.
7. Dos búsquedas dirigidas verifican producto a despacho y despacho a producto. Si falla cualquiera, los productos afectados se quitan del rack, del almacén y de los pedidos; luego se informa el problema con ProductoInaccesible.

Métodos principales

- add_door, add_dispatch_zone, add_rack, add_aisle, add_picker, add_product y add_order: incorporan entidades con las validaciones propias de cada tipo.
- validate_inside_warehouse y validate_element_bounds: verifican que un punto o el rectángulo completo de un elemento quede dentro de la grilla.
- _adjacent_cells_of y _rack_access_cells: calculan vecindades ortogonales de elementos y puntos de acceso a racks.
- generar_pasillos_faltantes y _particion_voraz_mixta: particionan el espacio libre en rectángulos; la elección del rectángulo de mayor área es voraz y local.
- build_graph y _agregar_arista: construyen el grafo completo dirigido, con una arista por movimiento permitido entre celdas vecinas.
- build_reduced_graph_v2 y _celdas_rectangulo: comprimen pasillos en tramos entre puntos relevantes y reconstruyen celdas de sectores eliminados.
- bloquear_pasillos_sin_salida: limpia sectores sin salida y modifica los grafos y pasillos en el lugar; el método no tiene un return explícito y devuelve None.
- validar_acceso_productos_a_despacho y validar_acceso_despacho_a_producto: devuelven los IDs de productos que no alcanzan el otro extremo.
- _eliminar_productos_inaccesibles: sincroniza la eliminación de productos en racks, pedidos e índices de accesos.

2. Algoritmos ejecutados al llamar a build_graph()

La llamada encadena la generación de pasillos faltantes, la creación del grafo por celdas y la construcción del grafo reducido. En esa última etapa se eliminan sectores sin salida y se validan los recorridos desde y hacia despacho. El proceso termina con el grafo reducido listo o con una excepción si hay productos inaccesibles.

2.1 Generación de pasillos faltantes

generar_pasillos_faltantes() y _particion_voraz_mixta() cubren con pasillos las celdas que todavía no pertenecen a racks, zonas de despacho o pasillos declarados.

1. Se combinan las celdas ocupadas de racks, zonas y pasillos. Con ellas se arma una grilla booleana de ancho W y alto H: libre significa que esa coordenada todavía no está ocupada.
2. _particion_voraz_mixta recorre la grilla de arriba abajo y de izquierda a derecha. Al encontrar una celda libre, la toma como esquina superior de inicio del rectángulo candidato.
3. Desde esa columna, inspecciona las filas siguientes mientras la celda de inicio siga libre. En cada fila cuenta las celdas libres contiguas hacia la derecha y conserva el menor ancho común visto hasta esa altura.
4. Para cada altura calcula área = ancho común × alto. Guarda el rectángulo de mayor área que pudo formar desde esa celda inicial.
5. Marca el rectángulo elegido como ocupado y continúa la búsqueda desde la columna siguiente a su borde. Así evita volver a particionar sus celdas.
6. generar_pasillos_faltantes crea un Aisle PAS_AUTO_n de sentido AMBOS por cada rectángulo y lo incorpora con add_aisle, que registra sus celdas y sus Vertice.

La elección es voraz: optimiza el rectángulo desde la primera celda libre encontrada, no la partición global de toda la grilla.

2.2 Construcción del grafo completo

build_graph() trabaja sobre las celdas de pasillo. Cada coordenada transitable se convierte en nodo y cada movimiento ortogonal permitido se representa como arista dirigida de costo 1.

1. Después de completar los pasillos faltantes, crea el índice coordenada → Vertice para encontrar en tiempo esperado constante el pasillo dueño de cada celda vecina.

2. Recorre cada pasillo y cada celda de ese pasillo. Para cada celda prueba cuatro destinos: una unidad abajo, arriba, a la izquierda y a la derecha.
3. Solo agrega un movimiento si el destino también es una celda de pasillo y los sentidos del pasillo de origen y del destino permiten el desplazamiento. El código excluye N para el movimiento hacia y-1, S para y+1, E para x-1 y O para x+1.
4. _agregar_arista guarda (destino, 1) en la lista del origen y asegura que ambos nodos estén representados en graph.
5. Si origen y destino pertenecen a pasillos distintos, registra la coordenada de origen en la salida del pasillo de origen y en la entrada del pasillo de destino. Las listas evitan duplicar la misma coordenada.
6. Al terminar de recorrer las celdas, build_graph llama a build_reduced_graph_v2 para derivar el grafo de planificación y validar el layout.

2.3 Reducción del grafo

build_reduced_graph_v2() comprime secuencias de movimientos rectos. Conserva como nodos los lugares donde conviene decidir o cambiar de tramo y guarda en las aristas la distancia real recorrida.

1. Si graph está vacío, solicita build_graph() y retorna. En la llamada normal desde build_graph, el grafo completo ya fue creado.
2. Inicializa los candidatos con los accesos cargados de productos y las cuatro esquinas geométricas de cada pasillo.
3. Revisa las esquinas que tienen conexiones registradas con otros pasillos. Si existe una arista real entre celdas vecinas de ambos pasillos, conserva los dos lados del contacto, para que el grafo reducido represente el cruce dirigido.
4. Para cada acceso de producto dentro de un pasillo, identifica el rack y proyecta una línea desde el acceso hacia el extremo de la cara del rack. Avanza mientras haya celdas de pasillo y aristas dirigidas válidas.
5. Procesa iterativamente cada candidato: recorre las cuatro direcciones hasta que se corte una arista o se termine el pasillo. Si atraviesa un contacto con otro pasillo, agrega los dos extremos del contacto; también agrega el extremo alcanzable del tramo. Los nuevos candidatos se procesan después.
6. Añade celdas de pasillo exteriores a las esquinas de cada zona de despacho. Omite las dos celdas del lado donde está ubicada la puerta.
7. Desde cada candidato vuelve a recorrer las cuatro direcciones. Al llegar a otro candidato agrega una arista al grafo reducido con distancia igual a la cantidad de pasos. Una interrupción de aristas termina ese rayo.
8. Guarda el resultado en reduced_graph y encadena el bloqueo de sectores y las dos validaciones de conectividad.

2.4 Detección y eliminación de sectores sin salida

bloquear_pasillos_sin_salida() busca sectores que un picker puede alcanzar al entrar a un pasillo pero desde los que no existe una ruta dirigida a ninguna salida registrada de ese pasillo.

1. Construye un índice de celda a pasillo propietario y agrupa por pasillo los nodos que todavía aparecen en el grafo reducido.
2. Para cada pasillo activo identifica salidas directas: una celda que figura en salida debe tener una arista reducida hacia un nodo cuyo dueño sea el pasillo destino.
3. Forma el grafo inverso interno del pasillo. Recorriendo predecesores desde las salidas directas, obtiene pueden_salir: los nodos que sí tienen camino hasta alguna salida.

4. Identifica las entradas registradas desde pasillos vecinos y confirma que haya una arista reducida desde la celda de origen al pasillo actual. Desde esos nodos recorre aristas hacia delante, limitándose al pasillo, para obtener alcanzables.
5. La diferencia alcanzables - pueden_salir son nodos atrapados: se puede llegar a ellos desde una entrada, pero no se puede continuar hasta una salida.
6. Agrupa los nodos atrapados por componente conectado dentro de su pasillo. Calcula sus límites; para pasillos alargados extiende el rectángulo a todo el ancho transversal del pasillo. _celdas_rectangulo materializa las coordenadas y el resultado se recorta a las celdas del pasillo.
7. Elimina las celdas del grafo completo, del reducido y de los índices de vértices y pasillos. También retira aristas que entran, salen o atraviesan esos sectores y filtra las relaciones entrada/salida que ya no tienen conexión activa.
8. Actualiza el índice de propietarios, marca blocked en los pasillos vaciados y repite el análisis. Las eliminaciones pueden desconectar otras secciones que antes sí parecían tener salida.

El ciclo termina cuando ya no quedan sectores atrapados. El método modifica las estructuras en el lugar y, como no tiene un return explícito, devuelve None.

2.5 Validación de accesibilidad

_nodos_despacho_en_grafo() reúne los nodos del grafo reducido que son vecinos ortogonales de las zonas de despacho. Las dos validaciones siguientes usan esos nodos como extremos de búsqueda y respetan la dirección de las aristas.

Producto hacia despacho

1. validar_acceso_productos_a_despacho inicializa como conectados los nodos adyacentes a despacho. Si no hay productos devuelve una lista vacía; si hay productos pero no hay zonas, lanza ValueError.
2. Para cada acceso de producto que todavía no tiene resultado conocido, ejecuta una BFS dirigida con cola, conjunto de visitados y mapa de padres. Un acceso ausente del grafo se marca como inaccesible.
3. Cuando alcanza un nodo ya conocido como conectado, propaga el resultado por el camino de padres y lo memoriza para reutilizarlo desde otros accesos. Si agota la búsqueda, marca como no conectados todos los nodos visitados.
4. Devuelve los IDs de los productos cuyo acceso no llegó a despacho.

Despacho hacia producto

1. validar_acceso_despacho_a_producto inicia una sola BFS multi-origen con todos los nodos adyacentes a las zonas de despacho.
2. Expande las aristas salientes y marca cada acceso de producto alcanzado. Termina cuando alcanza todos los accesos o cuando ya no quedan nodos por explorar.
3. Devuelve los IDs cuyos accesos no fueron alcanzados desde despacho.

build_reduced_graph_v2 combina los dos conjuntos de IDs: basta con que falle uno de los sentidos para considerar inaccesible un producto. _eliminar_productos_inaccesibles lo retira del rack y del diccionario de productos, elimina de los pedidos las líneas que lo referencian, quita pedidos vacíos y recalcula los accesos. Después se lanza ProductoInaccesible con el detalle de los productos afectados.

3. Uso en la carga del escenario

Paso a paso

1. cargar_warehouse(datos) crea Warehouse a partir de las dimensiones del escenario.

2. Carga puertas y zonas de despacho; después agrega racks y pasillos definidos, pickers y productos. Los productos se ordenan por rack y nivel para insertarlos desde abajo.
3. La clase mantiene las relaciones espaciales y asocia cada producto con el rack que lo contiene. Los pedidos se agregan en el modelo que consume la simulación.
4. La construcción de los grafos se solicita posteriormente mediante build_graph, que también genera los pasillos que falten y valida la conectividad del layout.

4. Consideraciones

- Las aristas son dirigidas: poder ir del producto al despacho no implica poder regresar del despacho al producto.
- Los setters de colecciones agregan elementos al estado actual; no vacían previamente el diccionario interno.
- La conectividad se valida al construir el grafo reducido. Si hay productos inaccesibles, la construcción elimina esos productos y lanza ProductoInaccesible.